Self-Evaluating Lesson Content Generator

Assessment Design Document | Role: GenAI Engineer - Content Systems | Topic: Introduction to RAG

1. Objective

An agentic generate, evaluate, regenerate loop that decides on its own whether a beginner lesson is good enough to ship, logs every rejection with what changed on retry, and learns across runs.

2. Architecture

Controller loads the source pack and memory hints. Generator writes the lesson. Evaluator (deterministic gates, plus an independent LLM judge in API mode) returns PASS/FAIL per check with reasons. Any FAIL sends the reasons back to the generator. Loop = 1 attempt + at most 2 retries, enforced in code. If nothing passes, output is marked NOT APPROVED. SQLite memory stores attempts, failure reasons, runs and human feedback.

3. Rubric v2.0.0 (hard pass/fail)

Check	Verifies
definition	RAG expanded and defined up front
why_it_matters	problem + how documents help
how_it_works	>=4 ordered steps: chunk, search, model
concrete_example	Question / Retrieved info / Answer; answer grounded in retrieved text
jargon_explained	every watched technical term is defined
beginner_language	avg sentence <=18 words, max <=32, 250-900 words, analogy
key_points	>=2 benefits, >=2 limits, keyword/vector search
coherent_structure	7 headings in required order
grounding	no overclaims, no invented numbers, explicit non-guarantee
clean_output	no leaked meta text
llm_* (API mode)	factual accuracy, clarity, example quality, flow

4. Retry and rejection log

Each rejection records failed checks, reasons, the feedback sent to the generator, and *changes_on_retry*: checks fixed, still failing, newly broken, word delta and diff samples. The demo simulator writes a flawed draft 1 (no labelled example, undefined jargon, an overclaim); the retry fixes only what the evaluator named. Sample run: rejected on attempt 1 (3 failed checks), approved on attempt 2.

5. Memory and self-evolution

Frequent failure reasons and human feedback (--feedback) are injected into later prompts. Checks failing 5 or more times are flagged for human rubric review. The rubric is never edited automatically, so the system cannot lower its own bar.

6. Trade-offs

Rules are transparent, cheap and tested but gameable; an LLM judge is flexible but inconsistent and may share blind spots, so use a hybrid with human approval. Two retries bound cost and guarantee termination. No vector DB: one topic, small source pack.

7. Validation

16 offline unit tests: good lesson passes; each failure type is caught; negated 'does not guarantee' is not flagged; retry cap holds at 3 attempts; API outage does not crash; memory persists across runs. Run: python -m unittest discover -s tests -v

8. Limitations

Deterministic checks cannot prove every fact. Next: per-claim source spans, a second judge model, learner testing, manual publish approval.